Lecture 6

Last time we saw Deutsch's algorithm that uses only 1 or 2 qubits. This time we will see extensions to n qubits of this basic algorithm. These extensions play around with the same idea in different ways. They are all based on the Hadamard transform, a generalization of the simple 2×2 Hadamard matrix $H=\frac{1}{\sqrt{2}}\begin{pmatrix}1&1\\1&-1\end{pmatrix}$.

Hadamard transform

The n-qubit Hadamard transform is

$$H^{\otimes n}=H\otimes H^{\otimes n-1}=\frac{1}{\sqrt{2}}\begin{pmatrix}1&1\\1&-1\end{pmatrix}\otimes H^{\otimes n-1}=\frac{1}{\sqrt{2}}\begin{pmatrix}H^{\otimes n-1}&H^{\otimes n-1}\\H^{\otimes n-1}&-H^{\otimes n-1}\end{pmatrix}.$$

For example, $H^{\otimes 2}=\frac{1}{2}\begin{pmatrix}1&1&1&1\\1&-1&1&-1\\1&1&-1&-1\\1&-1&-1&1\end{pmatrix}$.

Note that

$$H=\frac{1}{\sqrt{2}}\begin{pmatrix}1&1\\1&-1\end{pmatrix}=\sum_{x,y\in\{0,1\}}\frac{1}{\sqrt{2}}(-1)^{xy}|x\rangle\langle y|$$

Hence $H_{xy}=\frac{1}{\sqrt{2}}(-1)^{xy}$ for all $x,y\in\{0,1\}$.

Similarly,

$$H\otimes H=\left(\frac{1}{\sqrt{2}}\sum_{x_1,y_1\in\{0,1\}}(-1)^{x_1y_1}|x_1\rangle\langle y_1|\right)\otimes\left(\frac{1}{\sqrt{2}}\sum_{x_2,y_2\in\{0,1\}}(-1)^{x_2y_2}|x_2\rangle\langle y_2|\right)$$

$$=\frac{1}{2}\sum_{x_1,x_2,y_1,y_2\in\{0,1\}}(-1)^{x_1y_1+x_2y_2}|x_1\rangle\langle y_1|\otimes|x_2\rangle\langle y_2|=\frac{1}{2}\sum_{x_1,x_2,y_1,y_2\in\{0,1\}}(-1)^{x_1y_1+x_2y_2}|x_1x_2\rangle\langle y_1y_2|$$

so $\frac{1}{2}(-1)^{x_1y_1+x_2y_2}$ is the entry of $H^{\otimes 2}$ at row x_1x_2 and column y_1y_2.

By induction,

$$H^{\otimes n} = \sum_{x,y \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} (-1)^{x \cdot y} |x\rangle\langle y|$$

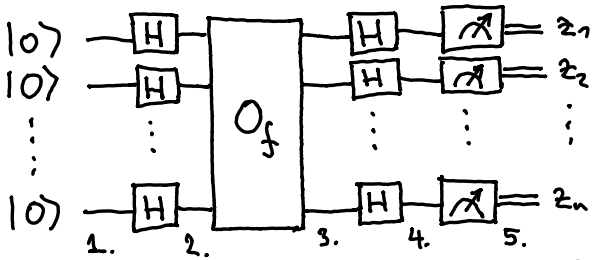
where $x \cdot y = \sum_{i=1}^{n} x_i y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n \pmod 2$ is the inner product of x and y modulo 2, i.e., as vectors in the vector space $\mathbb{Z}_2^n$. We see from this that

$$H^{\otimes n} |y\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot y} |x\rangle$$

for any computational basis state $|y\rangle$, $y \in \{0,1\}^n$.
Since $H \cdot H = I$, we also have

$$H^{\otimes n} \cdot \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot y} |x\rangle = |y\rangle \quad \text{for any } y \in \{0,1\}^n.$$

Deutsch-Jozsa algorithm (with phase oracle)



$O_f |x\rangle = (-1)^{f(x)} |x\rangle$

$z = z_1 z_2 \ldots z_n \in \{0,1\}^n$

1. $|0\rangle^{\otimes n}$
2. $(H|0\rangle)^{\otimes n} = |+\rangle^{\otimes n} = \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right)^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$ ← (Same as $H^{\otimes n}|0^n\rangle$)
3. $\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} O_f |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle$
4. $\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle = \sum_{z \in \{0,1\}^n} \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x) + z \cdot x} |z\rangle$
5. Get $z \in \{0,1\}^n$ with prob. $\left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x) + z \cdot x} \right|^2$

Deutsch-Jozsa problem

We get $z \in \{0,1\}^n$ with probability $p_z = \left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+z \cdot x} \right|^2$.

For example, the probability to get $z = 00\ldots0 = 0^n$ is $p_0 = \left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \right|^2$ where $\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)}$ is simply the average of $(-1)^{f(x)}$ over all $x \in \{0,1\}^n$. When is $p_0 = 1$ or $p_0 = 0$?

- $p_0 = 1$ if $\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} = \pm 1$, meaning that f is <u>constant</u>,

  i.e., either $f(x) = 0$ for all $x \in \{0,1\}^n$
  or $f(x) = 1$ for all $x \in \{0,1\}^n$.

- $p_0 = 0$ if $\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} = 0$, meaning that f is <u>balanced</u>,

  i.e., $|\{x \in \{0,1\}^n : f(x) = 0\}| = |\{x \in \{0,1\}^n : f(x) = 1\}| = 2^n/2$.

Conclusion: if f is constant, we always get outcome $z = 0^n$, if f is balanced, we never get $z = 0^n$.

<u>Problem</u>: promised that f is either constant or balanced, decide which of the two it is by evaluating f on inputs of your choice.

<u>Result</u>: A quantum algorithm can solve this with certainty by evaluating f only once. A deterministic classical algorithm needs $2^{n-1}+1$ evaluations, which is significantly more. However, a probabilistic classical algorithm can solve this with probability $\frac{3}{4}$ by evaluating f only twice (homework...).

Bernstein-Vazirani problem

Recall that the Deutsch-Jozsa algorithm outputs $z \in \{0,1\}^n$ with probability $p_z = \left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+z \cdot x} \right|^2$.

What other interesting functions can we choose so that p_z is always either 1 or 0? Let $f(x) = s \cdot x$ for some $s \in \{0,1\}^n$, i.e., f is linear over $\mathbb{Z}_2$. Then

$$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+z \cdot x} = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{(s+z) \cdot x}$$

Note that

$$\sum_{x \in \{0,1\}^n} (-1)^{00\ldots0 \cdot x} = \sum_{x \in \{0,1\}^n} 1 = 2^n$$

so $p_s = \left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{(s+s) \cdot x} \right|^2 = \left| \frac{1}{2^n} \cdot 2^n \right|^2 = |1|^2 = 1.$

Since p is obtained by measuring a quantum state, it is a probability distribution, implying that $p_z = 0$ for $z \neq s$ (exercise: verify this explicitly).

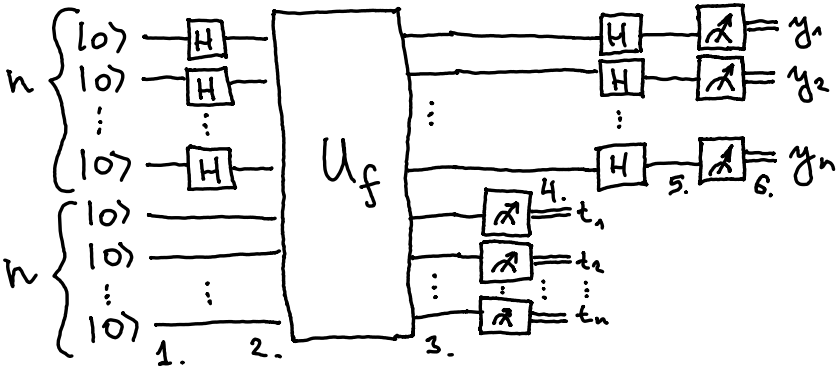
Conclusion: if $f(x) = s \cdot x$, for some $s \in \{0,1\}^n$, then $p_s = 1$ and $p_z = 0$ for any $z \neq s$. Hence, the measurement outcome is always $z = s$.

Problem: promised that $f(x) = s \cdot x$, for some $s \in \{0,1\}^n$, determine s by evaluating f on inputs of your choice.

Result: A quantum algorithm can solve this problem with certainty by evaluating f once. Any deterministic algorithm needs to evaluate f at least n times (exercise). Even a probabilistic algorithm needs to evaluate f $O(n)$ times to succeed with probability $\geq 3/4$.

Simon's algorithm

Consider a more general function $f:\{0,1\}^n \to \{0,1\}^n$. Let $U_f|x,y\rangle = |x, y \oplus f(x)\rangle$ be the bit oracle for f where $x, y \in \{0,1\}^n$. Let's try a similar procedure:



1. $|0\rangle^{\otimes n}|0\rangle^{\otimes n}$
2. $|+\rangle^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle|0^n\rangle$
3. $\frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}|x\rangle|f(x)\rangle$
4. If we get outcome $t = t_1 t_2 \ldots t_n \in \{0,1\}^n$ and let $F_t = \{x : f(x) = t\}$ then the remaining state on the first n qubits is

$$\frac{1}{\sqrt{|F_t|}}\sum_{x\in F_t}|x\rangle \leftarrow \begin{cases}\text{a superposition of all } x \text{ such} \\ \text{that } f(x)=t\end{cases}$$

Let's assume there are exactly two x such that $f(x)=t$. I.e., $F_t = \{x, x+a\}$ for some $x, a \neq 0 \in \{0,1\}^n$. Then the state on the first n qubits is $\frac{1}{\sqrt{2}}(|x\rangle + |x+a\rangle)$.

5. $H^{\otimes n}\cdot\frac{1}{\sqrt{2}}(|x\rangle+|x+a\rangle) = \frac{1}{\sqrt{2}}\cdot\frac{1}{\sqrt{2^n}}\sum_{y\in\{0,1\}^n}\left((-1)^{y\cdot x}+(-1)^{y\cdot(x+a)}\right)|y\rangle$

$$= \frac{1}{\sqrt{2^{n+1}}}\sum_{y\in\{0,1\}^n}(-1)^{y\cdot x}\underbrace{\left(1+(-1)^{y\cdot a}\right)}_{0 \text{ or } 2}|y\rangle = \frac{1}{\sqrt{2^{n+1}}}\sum_{y:y\cdot a=0}(-1)^{y\cdot x}\,2\,|y\rangle$$

keep only non-zero terms

$$= \frac{1}{\sqrt{2^{n-1}}}\sum_{y:y\cdot a=0}(-1)^{y\cdot x}|y\rangle \leftarrow \text{a superposition of all } y \text{ such that } y\cdot a = 0, \text{ i.e., } y \perp a$$

6. When we measure $\frac{1}{\sqrt{2^{n-1}}}\sum_{y:y\cdot a=0}(-1)^{y\cdot x}|y\rangle$ in the computational basis, the phases $(-1)^{y\cdot x}$ do not play any role, so we simply get a uniformly random $y\in\{0,1\}^n$ such that $y\cdot a=0$. This gives us information about a.

Note that for any given $a\neq 0^n$ half of the strings $y\in\{0,1\}^n$ are such that $y\cdot a=0$ and half of them are such that $y\cdot a=1$ (just change the i-th bit of y for any $i\in\{1,\dots,n\}$ such that $a_i=1$ to modify the value of $y\cdot a$). Hence the number of strings $y\in\{0,1\}^n$ such that $y\cdot a=0$ is indeed $2^n/2=2^{n-1}$.

If we want the above procedure to always work, we need to make sure $|F_t|=2$ for any $t\in\{0,1\}^n$. I.e., we want the function $f:\{0,1\}^n\to\{0,1\}^n$ evaluated by the oracle to be 2-to-1. Moreover, if we want to learn information about the same $a\in\{0,1\}^n$, we want $f(x)=f(x+a)$ for all $x\in\{0,1\}^n$, meaning that $F_t=\{x: f(x)=t\}=\{x_t, x_t+a\}$ where $x_t\in\{0,1\}^n$ depends on t but a does not.

<u>Assumption:</u> There exists an $a\neq 0^n$ such that $f(x)=f(x')\iff x'-x=a$

x	$f(x)$
00	11
01	11
10	01
11	01

$a=01$

This means that f is invariant under a shift by some string $a\in\{0,1\}^n$, $a\neq 0^n$, and it takes different values on all pairs of arguments that do not differ by a.

<u>Conclusion</u>: With one evaluation of such f, a quantum algorithm can find a uniformly random y that is orthogonal to a, i.e., $y\cdot a=0$.

If we execute this quantum subroutine many times (roughly n), we get enough vectors orthogonal to a that they in fact uniquely determine a.
So here is the problem the algorithm solves:

Problem: promised that $f:\{0,1\}^n \to \{0,1\}^n$ is such that $f(x)=f(x')$ if and only if $x-x'=a$, for some unknown $a\in\{0,1\}^n$, $a\neq 0^n$, determine a by evaluating f on inputs of your choice.

Here is the full Simon's algorithm, including the classical post-processing of the measurement outcomes.

Algorithm

1. For $i=1$ to $n+3$ do
   Execute the quantum subroutine to get a uniformly random $y_i\in\{0,1\}^n$ such that $y_i\cdot a=0$.
2. Using Gaussian elimination modulo 2 (or any other method) solve classically the linear system to find $a\in\{0,1\}^n$:
$$\begin{cases} y_1\cdot a=0 \\ y_2\cdot a=0 \\ \vdots \\ y_{n+3}\cdot a=0 \end{cases}$$
3. If it has a unique solution, output it. Otherwise output "FAIL".

Result: The above quantum algorithm evaluates f $n+3$ times and finds a with probability $\geq \frac{2}{3}$.
Any probabilistic classical algorithm needs to evaluate f at least $\sim 2^{n/3}$ times to find a with probability $\geq \frac{2}{3}$.

Comparison of the three algorithms

Below is a table comparing the complexities of the three problems we discussed in the class. In each case we list the number of queries to the function f and the success probability of the algorithm. We compare deterministic, probabilistic, and quantum algorithms. We would like to achieve a large gap between quantum and probabilistic algorithms — this would show that quantum computers are indeed more powerful. We don't care about the exact value of the success probability, as long as it is strictly larger than $1/2$. We can amplify it arbitrarily close to 1 by repeating the algorithm a constant number of times.

	Deutsch-Jozsa		Bernstein-Vazirani		Simon	
	Queries	Prob.	Queries	Prob.	Queries	Prob.
Deterministic	$2^{n-1}+1$	1	n	1	$2^{n-1}+1$	1
Probabilistic	2	3/4	$O(n)$	$\geq 3/4$	$2^{n/3}$	$\geq 2/3$
Quantum	1	1	1	1	$n+3$	$\geq 2/3$

Simon's algorithm requires exponentially fewer queries compared to the best possible probabilistic algorithm.